Groth16 설정 단계 (Setup Phase) 상세 분석

목차

1. 설정 단계 개요
2. 독성 폐기물 생성
3. 회로 합성 및 분석
4. R1CS에서 QAP로 변환
5. 키 구성 요소 계산
6. 배치 다중 스칼라 곱셈
7. 보안 고려사항
8. 성능 최적화

설정 단계 개요

설정 단계는 Groth16 시스템의 가장 중요하고 민감한 부분입니다. 이 단계에서 생성되는 **구조화된 참조 문자열(Structured Reference String, SRS)**이 전체 시스템의 보안성을 결정합니다.

전체 워크플로우

```
pub fn generate_random_parameters_with_reduction<C>(
circuit: C,
rng: &mut impl Rng,
) -> R1CSResult<ProvingKey<E>>
```

```
where
C: ConstraintSynthesizer<E::ScalarField>,
{
// 1. 독성 폐기물 생성
let alpha = E::ScalarField::rand(rng);
let beta = E::ScalarField::rand(rng);
let gamma = E::ScalarField::rand(rng);
let delta = E::ScalarField::rand(rng);

// 2. 그룹 생성원 선택
let g1_generator = E::G1::rand(rng);
let g2_generator = E::G2::rand(rng);

// 3. 실제 키 생성
Self::generate_parameters_with_qap(
circuit, alpha, beta, gamma, delta,
g1_generator, g2_generator, rng,
)
}
```

신뢰할 수 있는 설정의 중요성

왜 "독성 폐기물"인가?

- 이 비밀 값들을 아는 사람은 **가짜 증명을 생성**할 수 있습니다
- 설정 완료 후 이 값들은 **완전히 삭제**되어야 합니다
- 실제 운영에서는 **다자간 계산(MPC)**을 통해 생성합니다

독성 폐기물 생성

비밀 값들의 역할

```
// 4개의 핵심 비밀 값
let alpha = E::ScalarField::rand(rng); // A 원소의 구조적 무결성
let beta = E::ScalarField::rand(rng); // B 원소의 구조적 무결성
let gamma = E::ScalarField::rand(rng); // 공개/비밀 입력 분리
let delta = E::ScalarField::rand(rng); // 영지식성 보장
```

각 비밀 값의 수학적 역할:

1. **α (alpha)**:
   - A 원소에 구조를 부여: `A = α + Σ(a_i * u_i) + r * δ`
   - 페어링 방정식의 좌변에서 사용: `e(A, B)`
   - 지식 추출 가능성 보장
2. **β (beta)**:
   - B 원소에 구조를 부여: `B = β + Σ(b_i * u_i) + s * δ`
   - 페어링 방정식의 우변에서 사용: `e(α, β)`
   - A와 B의 일관성 보장
3. **γ (gamma)**:

- 공개 입력 처리: `γ^(-1) * (β*a_i + α*b_i + c_i)`
- 공개/비밀 정보 분리
- 검증자가 공개 입력만으로 검증 가능하게 함

4. **δ (delta)**:

- witness 부분 처리: `δ^(-1) * (β*a_i + α*b_i + c_i)`
- 영지식성 보장: 무작위 값 r, s와 결합
- QAP 몫 다항식 처리

그룹 생성원 선택

```
let g1_generator = E::G1::rand(rng);
let g2_generator = E::G2::rand(rng);
```

무작위 생성원 사용 이유:

- **Nothing-up-my-sleeve**: 특별한 구조가 없는 무작위 점 사용
- **보안성 향상**: 알려진 이산로그 관계 방지
- **투명성**: 생성 과정의 투명성 보장

실제 운영 환경에서의 생성

```
// 실제로는 이렇게 하면 안됨!
// 단일 엔티티가 모든 비밀을 알게 됨
let secrets = generate_secrets_single_party(rng);

// 올바른 방법: 다자간 계산 (MPC)
```

```
let secrets = generate_secrets_mpc(participants, threshold);
```

MPC 기반 설정의 특징:

- **분산 신뢰**: 여러 참가자가 협력하여 생성
- **임계값 보안**: k명 이상이 협력해야 비밀 복원 가능
- **검증 가능성**: 각 참가자가 다른 참가자의 기여도 검증

회로 합성 및 분석

제약 시스템 초기화

```
let setup_time = start_timer!(|| "Groth16::Generator");

let cs = ConstraintSystem::new_ref();

cs.set_optimization_goal(OptimizationGoal::Constraints);

cs.set_mode(SynthesisMode::Setup);
```

설정 모드의 특징:

- **구조만 생성**: 실제 witness 값 없이 제약 조건 구조만 파악
- **최적화 목표**: 제약 조건 수 최소화에 집중
- **메모리 효율성**: 불필요한 witness 저장 생략

회로 합성 과정

```
// 회로의 제약 조건들을 생성

let synthesis_time = start_timer!(|| "Constraint synthesis");
```

```
circuit.generate_constraints(cs.clone())?;
end_timer!(synthesis_time);


// 선형 결합 최적화
let lc_time = start_timer!(|| "Inlining LCs");
cs.finalize();
end_timer!(lc_time);
```

합성 과정의 세부 단계:

1. **변수 할당**: 공개 입력과 witness 변수들 정의
2. **제약 조건 생성**: 각 게이트에 대한 R1CS 제약 조건 생성
3. **선형 결합 인라이닝**: 중간 변수들을 제거하여 최적화
4. **최종 검증**: 제약 조건의 일관성 확인

도메인 구성

```
let domain_time = start_timer!(|| "Constructing evaluation domain");


// 도메인 크기 = 제약 조건 수 + 공개 입력 수
let domain_size = cs.num_constraints() + cs.num_instance_variables();
let domain = D::new(domain_size).ok_or(SynthesisError::PolynomialDegreeTooLarge


// 도메인 외부의 무작위 점 선택
let t = domain.sample_element_outside_domain(rng);
```

```
end_timer!(domain_time);
```

도메인 설계 원칙:

1. **2의 거듭제곱 크기**: FFT 효율성을 위해
2. **충분한 크기**: 모든 제약 조건과 공개 입력 수용
3. **외부 점 선택**: 보안성을 위해 도메인 외부의 점 τ 사용

도메인 크기 계산 예시:

```
// 예: 100개 제약 조건, 5개 공개 입력
let domain_size = 100 + 5; // 105
let domain = D::new(105); // 실제로는 128 (2^7)로 확장
```

R1CS에서 QAP로 변환

변환 과정 개요

```
let reduction_time = start_timer!(|| "R1CS to QAP Instance Map with Evaluation"
let num_instance_variables = cs.num_instance_variables();
let (a, b, c, zt, qap_num_variables, m_raw) =
QAP::instance_map_with_evaluation::<E::ScalarField, D<E::ScalarField>>(cs, &t)?;
end_timer!(reduction_time);
```

변환 결과물:

- `a`, `b`, `c` : 각 변수에 대한 QAP 다항식 계수들
- `zt` : 소멸 다항식 Z(t)의 값
- `qap_num_variables` : QAP 변수 개수
- `m_raw` : 도메인의 실제 크기

라그랑주 보간 과정

```
// r1cs_to_qap.rs에서
fn instance_map_with_evaluation<F: PrimeField, D: EvaluationDomain<F>>(
cs: ConstraintSystemRef<F>,
t: &F,
) -> R1CSResult<(Vec<F>, Vec<F>, Vec<F>, F, usize, usize)> {
        let matrices = cs.to_matrices().unwrap();
        let domain = D::new(domain_size)?;
        // 소멸 다항식 Z(t) = Π(t - ω^i) 계산
        let zt = domain.evaluate_vanishing_polynomial(*t);
        // 모든 라그랑주 계수 L_i(t) 계산
        let coefficients_time = start_timer!(|| "Evaluate Lagrange coefficients
        let u = domain.evaluate_all_lagrange_coefficients(*t);
        end_timer!(coefficients_time);
        // QAP 다항식 계수들 초기화
        let mut a = vec![F::zero(); qap_num_variables + 1];
        let mut b = vec![F::zero(); qap_num_variables + 1];
        let mut c = vec![F::zero(); qap_num_variables + 1];
```

```
// 공개 입력 부분 설정
{
        let  start = 0;
        let  end = cs.num_instance_variables();
        let  num_constraints = cs.num_constraints();
        a[start..end].copy_from_slice(&u[(start + num_constraints)..(end
}
// 각 제약 조건에 대해 다항식 계수 계산
for (i, u_i) in  u.iter().enumerate().take(cs.num_constraints()) {
        // A_j(x) = Σ(a_{i,j} * L_i(x))
        for &(ref  coeff, index) in &matrices.a[i] {
                a[index] += &(*u_i * coeff);
        }
        // B_j(x) = Σ(b_{i,j} * L_i(x))
        for &(ref  coeff, index) in &matrices.b[i] {
                b[index] += &(*u_i * coeff);
        }
        // C_j(x) = Σ(c_{i,j} * L_i(x))
        for &(ref  coeff, index) in &matrices.c[i] {
                c[index] += &(*u_i * coeff);
        }
}
Ok((a, b, c, zt, qap_num_variables, domain.size()))
```

```
}
```

수학적 의미:

1. **라그랑주 기저**: `L_i(ω^j) = δ_{i,j}` (크로네커 델타)
2. **다항식 보간**: `A_j(x) = Σ(a_{i,j} * L_i(x))`
3. **점 평가**: 점 t에서 모든 다항식 계수 계산

소멸 다항식의 역할

```
// Z(x) = Π(x - ω^i) for i = 0, ..., n-1
let  zt = domain.evaluate_vanishing_polynomial(*t);
```

소멸 다항식의 특성:

- **도메인에서 0**: 모든 ω^i에서 Z(ω^i) = 0
- **도메인 외부에서 비영**: t ∉ domain이면 Z(t) ≠ 0
- **나눗셈 가능성**: A(x)*B(x) - C(x)가 Z(x)로 나누어떨어지는지 확인

키 구성 요소 계산

역원 계산 및 검증

```
// 역원들 계산 (0이면 에러)
let  gamma_inverse = gamma.inverse().ok_or(SynthesisError::UnexpectedIdentity)?;
let  delta_inverse = delta.inverse().ok_or(SynthesisError::UnexpectedIdentity)?;
```

역원 계산의 중요성:

- **수학적 필요성**: γ^(-1), δ^(-1) 항들이 키 생성에 필수
- **보안 검증**: 0이면 시스템이 안전하지 않음
- **에러 처리**: 역원이 존재하지 않으면 즉시 중단

공개 입력용 계수 계산

```
// γ^(-1) * (β*a_i + α*b_i + c_i) 계산
let  gamma_abc = cfg_iter!(a[..num_instance_variables])
.zip(&b[..num_instance_variables])
.zip(&c[..num_instance_variables])
.map(|((a, b), c)| {
// 선형 결합 계산
let  linear_combination = beta * a + &(alpha * b) + c;
// γ^(-1)로 나누기
linear_combination * &gamma_inverse
})
.collect::<Vec<_>>();
```

수학적 의미:

- **선형 결합**: 각 공개 입력에 대한 A, B, C 다항식의 결합
- **정규화**: γ로 나누어 검증 시 일관성 보장
- **효율성**: 검증 시 단일 MSM으로 모든 공개 입력 처리

witness용 계수 계산

```
// δ^(-1) * (β*a_i + α*b_i + c_i) 계산 (witness 변수들에 대해)
let l = cfg_iter!(a[num_instance_variables..])
.zip(&b[num_instance_variables..])
.zip(&c[num_instance_variables..])
.map(|((a, b), c)| {
let linear_combination = beta * a + &(alpha * b) + c;
linear_combination * &delta_inverse
})
.collect::<Vec<_>>();
```

공개 입력과의 차이점:

- **다른 정규화**: δ^(-1) 사용 (γ^(-1) 대신)
- **영지식성**: δ와 무작위 값들의 결합으로 영지식성 보장
- **witness 전용**: 비밀 witness 변수들에만 적용

배치 다중 스칼라 곱셈

전처리 테이블 생성

```
// G1 그룹용 전처리 테이블
let g1_window_time = start_timer!(|| "Compute G1 window table");
let num_scalars = non_zero_a + non_zero_b + qap_num_variables + m_raw + 1;
let g1_table = BatchMulPreprocessing::new(g1_generator, num_scalars);
end_timer!(g1_window_time);
```

```
// G2 그룹용 전처리 테이블
let g2_time = start_timer!(|| "Compute G2 table");
let g2_table = BatchMulPreprocessing::new(g2_generator, non_zero_b);
end_timer!(g2_time);
```

전처리 테이블의 구조:

```
// 윈도우 방법을 위한 미리 계산된 점들
// table[i] = 2^(i*w) * G (w는 윈도우 크기)
struct BatchMulPreprocessing<G: CurveGroup> {
        base_table: Vec<Vec<G::Affine>>, // 각 윈도우별 테이블
        window_size: usize, // 윈도우 크기
}
```

쿼리 계산 과정

```
// A 쿼리: a_i(τ) * G
let a_time = start_timer!(|| "Calculate A");
let a_query = g1_table.batch_mul(&a);
drop(a); // 메모리 절약을 위한 조기 해제
end_timer!(a_time);

// B 쿼리 (G1): b_i(τ) * G
```

```
let b_g1_time = start_timer!(|| "Calculate B G1");
let b_g1_query = g1_table.batch_mul(&b);
end_timer!(b_g1_time);


// B 쿼리 (G2): b_i(τ) * H
let b_g2_time = start_timer!(|| format!("Calculate B G2 of size {}", b.len()));
let b_g2_query = g2_table.batch_mul(&b);
drop(g2_table); // G2 테이블은 더 이상 불필요
end_timer!(b_g2_time);
```

H 쿼리 특별 처리

```
// H 쿼리: h_i(τ) * G = (τ^i * Z(τ) / δ) * G
let h_time = start_timer!(|| "Calculate H");
let h_scalars = QAP::h_query_scalars::<_, D<E::ScalarField>>(
m_raw - 1, // 최대 차수
t, // 평가점 τ
zt, // Z(τ) 값
delta_inverse // δ^(-1)
)?;
let h_query = g1_table.batch_mul(&h_scalars);
end_timer!(h_time);
```

H 쿼리 스칼라 생성:

```
// r1cs_to_qap.rs에서
fn h_query_scalars<F: PrimeField, D: EvaluationDomain<F>>(
max_power: usize,
t: F,
zt: F,
delta_inverse: F,
) -> Result<Vec<F>, SynthesisError> {
        let scalars = cfg_into_iter!(0..max_power)
        .map(|i| {
                // τ^i * Z(τ) / δ 계산
                zt * &delta_inverse * &t.pow([i as u64])
        })
        .collect::<Vec<_>>();
        Ok(scalars)
}
```

L 쿼리 계산

```
// L 쿼리: l_i * G (witness 변수들에 대해)
let l_time = start_timer!(|| "Calculate L");
let l_query = g1_table.batch_mul(&l);
drop(l); // 메모리 해제
```

```
end_timer!(l_time);
```

보안 고려사항

독성 폐기물 처리

```
// 실제 구현에서는 이런 식으로 처리해야 함
impl Drop for ToxicWaste {
    fn drop(&mut self) {
        // 메모리를 0으로 덮어쓰기
        self.alpha.zeroize();
        self.beta.zeroize();
        self.gamma.zeroize();
        self.delta.zeroize();
        // 추가적인 보안 조치
        for _ in 0..3 {
            // 여러 번 덮어쓰기
            self.overwrite_with_random();
        }
    }
}
```

보안 삭제 요구사항:

- **즉시 삭제**: 키 생성 완료 즉시 비밀 값들 삭제
- **안전한 덮어쓰기**: 메모리를 무작위 값으로 여러 번 덮어쓰기
- **하드웨어 지원**: 가능하면 HSM이나 SGX 사용

사이드 채널 공격 방지

```
// 상수 시간 연산 사용
fn constant_time_multiply(a: &F, b: &F) -> F {
    // 분기 없는 곱셈 구현
    a * b  // 실제로는 더 복잡한 구현 필요
}

// 타이밍 공격 방지
fn secure_random_generation(rng: &mut impl CryptoRng) -> F {
    loop {
        let candidate = F::rand(rng);
        if !candidate.is_zero() {
            return candidate;
        }
        // 타이밍을 일정하게 유지하기 위한 더미 연산
        let _ = F::rand(rng);
    }
}
```

검증 가능한 설정

```
// 설정 과정의 투명성을 위한 로깅
pub struct SetupTranscript {
    pub participants: Vec<ParticipantId>,
    pub contributions: Vec<ContributionProof>,
    pub final_keys: KeyCommitment,
}

impl SetupTranscript {
    pub fn verify(&self) -> bool {
        // 각 참가자의 기여도 검증
        // 최종 키의 정확성 검증
        // 독성 폐기물이 실제로 삭제되었는지 확인
        }
}
```

성능 최적화

병렬 처리 최적화

```
// 조건부 병렬 처리
let gamma_abc = cfg_iter!(a[..num_instance_variables])
```

```
.zip(&b[..num_instance_variables])

.zip(&c[..num_instance_variables])

.map(|((a, b), c)| {

        // CPU 집약적 연산을 병렬로 처리

        (beta * a + &(alpha * b) + c) * &gamma_inverse

})

.collect::<Vec<_>>();
```

병렬화 전략:

- **데이터 병렬성**: 독립적인 계산들을 여러 스레드에서 처리
- **임계값 기반**: 작은 작업은 순차 처리로 오버헤드 방지
- **메모리 지역성**: 캐시 효율성을 고려한 데이터 분할

메모리 사용량 최적화

```
// 단계별 메모리 해제

let  a_query = g1_table.batch_mul(&a);

drop(a); // a 벡터 즉시 해제


let  b_g1_query = g1_table.batch_mul(&b);

let  b_g2_query = g2_table.batch_mul(&b);

drop(b); // b 벡터 해제

drop(g2_table); // G2 테이블 해제
```

```
// 최종 정리

drop(g1_table);
```

메모리 관리 원칙:

- **조기 해제**: 더 이상 필요하지 않은 데이터 즉시 해제
- **단계별 처리**: 전체 데이터를 한 번에 메모리에 올리지 않음
- **재사용**: 가능한 경우 버퍼 재사용

수치적 안정성

```
// 역원 계산 시 안정성 확인

let  gamma_inverse = gamma.inverse().ok_or_else(|| {

eprintln!("Gamma is zero! This should never happen in a secure setup.");

SynthesisError::UnexpectedIdentity

})?;


// 오버플로우 방지

let  result = a.checked_mul(&b).ok_or(SynthesisError::Overflow)?;
```

수치적 안정성 보장:

- **역원 존재성**: 모든 역원 계산 전에 0 여부 확인
- **오버플로우 검사**: 큰 수 연산 시 오버플로우 방지
- **정밀도 유지**: 부동소수점 연산 시 정밀도 손실 최소화

설정 단계는 Groth16 시스템의 보안성과 효율성을 결정하는 가장 중요한 부분입니다. 이 단계에서 생성된 키들이 이후 모든 증명과 검증의 기반이 되므로, 수학적 정확성과 보안성을 동시에 보장하는 것

이 핵심입니다.